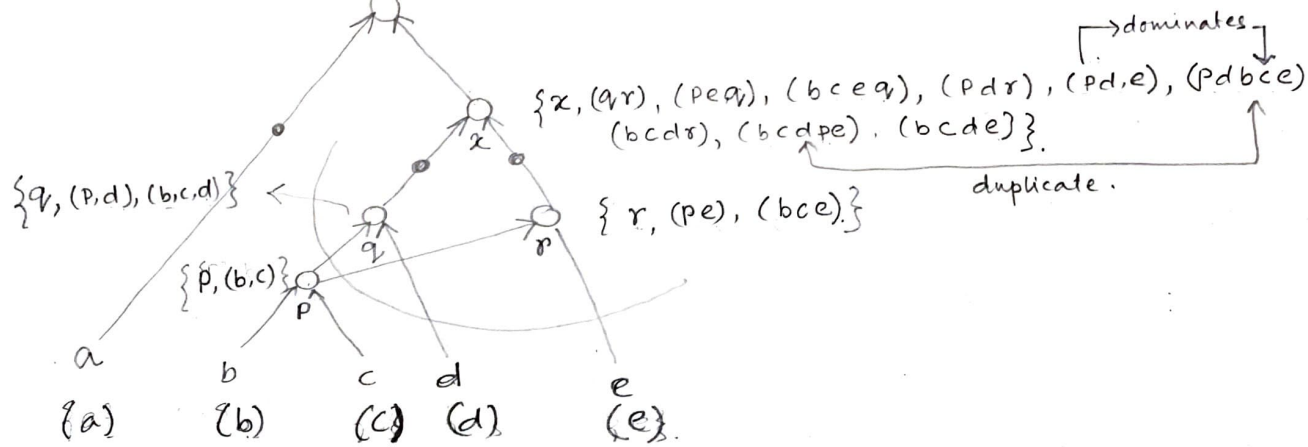


ASIC TM of ABC

1) Cut enumeration

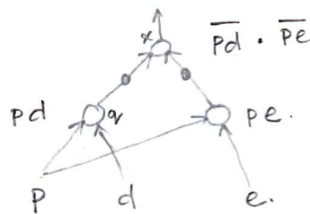
A cut is said to be dominated if there is another cut of the same node, which is contained, set theoretically, in the given cut.



2) Compute truth tables of cuts :-

Compute local function of a node in terms of its cut.

Ex:- for node x and its cut pde.



p	d	e	x
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

- Formal variables are assigned to each cut.
- ABC considers 5-feasible cuts.
- A 32-bit integer can be used to represent the TT.

3) Boolean Matching:-

Traditional solⁿ:- Two Boolean f^s are NPN equivalent if one can be transformed into the other by permuting or negating the inputs or negating the outputs.

- NPN-canonical representatives of the library gates are computed.
- Library gates are stored in a hash table indexed by the representatives.
- During matching the NPN-canonical representative is computed for every cut function. It is used to look up the hash table to find the appropriate gate.

Drawbacks of traditional Boolean Matching:-

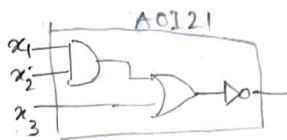
Requires computation of the NPN-representative for every cut function. Once the appropriate gate (from lib) is found, the appropriate variable correspondence must be found b/w the library gate and the cut function. These computations are done in the inner body of the mapper.

What ABC does?

Observation:- ABC considers only 5-feasible cuts i.e., functions have only 5-vars.

Precompute all functions obtained by permuting the inputs to library gates and add those to the hash table. Therefore, the cut function can be used directly to look the hash table.

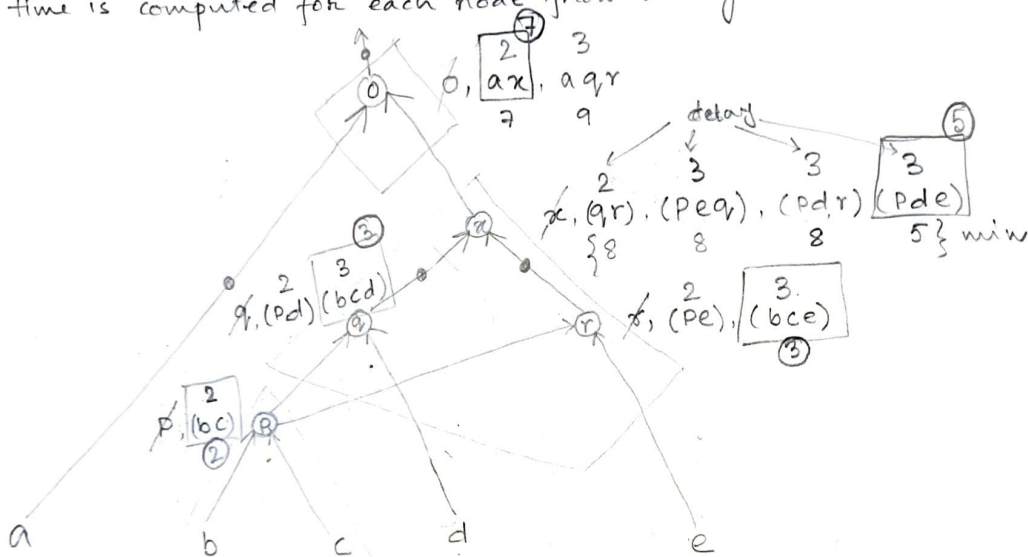
Ex:- AOI gate $(x_1, x_2 + x_3)$



$$\overline{x_1 x_3 + x_2}, \overline{x_2 x_3 + x_1}$$

4) Compute best arrival time at each node

Starting from the PI, and working in topological order towards the outputs, the best arrival time is computed for each node from amongst all its matches.



5) Choose the best cover

In the reverse topological order, the best gate for each PI is chosen. Next the best gates implementing the inputs of these gates are chosen and so on until all PIs have been reached.

Structural bias in TM

TM based on DAG covering suffers from the problem of structural bias i.e., the structure of the mapped netlist depends strongly on the subject graph.

Mitigate structural bias via lossless synthesis.

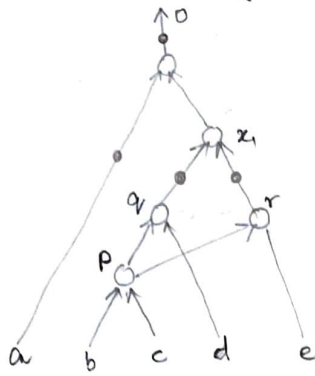
IDEA: Remember every network seen during a synthesis flow on a set of flows and to select the best part of each n/w during TM.

Why it is useful?

- Tech independent synthesis algorithms are heuristic (no optimality guarantee) in nature. By using only final n/w for TM, we might miss out on a better result that could be obtained from an intermediate n/w in the optimization flow.
- A flow to optimize delay might significantly increase area. By combining delay and area optimization, it is possible to get the best of both. For instance, on the critical path, the mapper can choose from delay-optimized n/w, whereas off critical path, the mapper can choose from area-optimized n/w.

Constructing the choice network:- Constructed from a collection of n/ws that are functionally equivalent.

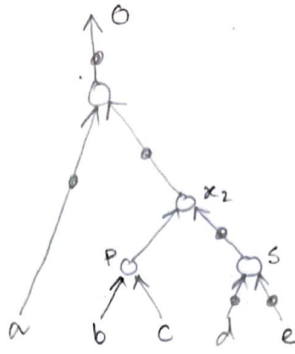
- Use BDD or (random simulation + SAT) - like FRAIG



$$P = bc$$

$$O = a + Pd + Pe$$

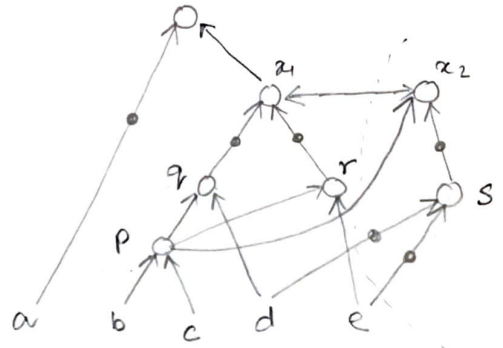
$\oplus$



$$P = bc$$

$$O = a + P(d + e)$$

$\Rightarrow$



n/w with choices

[q_1 and q_2 are functionally equivalent (upto complementation)]

Mapping with choices Only cut computation step needs modification.

$$\text{Cuts}(x) = \text{Cuts}(x_1) \cup \text{Cuts}(x_2)$$

Ex:- Compute 3-feasible cuts of O .

$$\phi_{\approx}(x) = \phi(x_1) \cup \phi(x_2).$$

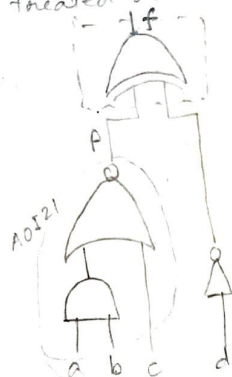
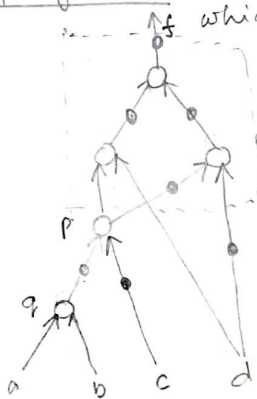
$$= \{x_1, x_2, (qr), (ps), (ape), (pdr), (pde), (bcs)\}$$

$$\phi_{\approx}(a) = \phi(a) = a$$

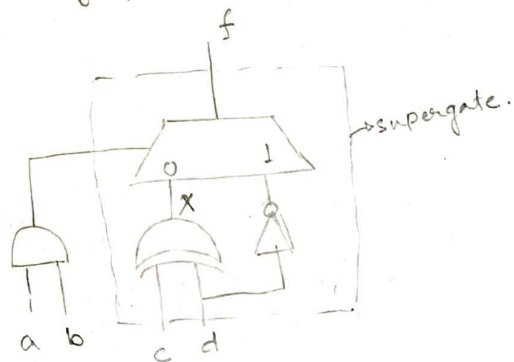
$$\phi_{\approx}(o) = \phi(o) = \{o \cup \{u \cup v \mid u \in \phi_{\approx}(a_x), v \in \phi_{\approx}(x_1), |u \cup v| \leq 3\}$$

$$\phi_{\approx}(o) = \{o, (ax_1), (ax_2), (aq_1r), (aps)\}$$

Supergates:- A supergate is a single op combinational n/w of a few library gates which is treated as a single library gate.



(a) Easy match



(b) Hard match

Note that one of the ips of the mux (x) is not present in the subject graph. Hence the mux is not a valid match for f. In contrast, the ips of XOR (P, d) are present in the subject graph. Hence XOR is a valid match for f.

Now if we connect the MUX, XOR and NOT together to form a new gate, it can be verified that this new gate is a match at f in the subject graph.

Hence, using supergates we can find better matches.

As seen in the last example, supergates match larger portion of the subject graph than library gates. This makes the matching more Boolean and less dependent on the structure of the subject graph. This also increases the number of matches seen by the mapper, and leads to better results.

- Use of supergate requires no change in the mapping procedure. Since supergates can be treated as ordinary library gates.
- The supergates are recomputed only when the technology library changes.

How to generate supergates?

Supergates (SGs) are generated recursively in a number of rounds.

Suppose K be the max no. of ips in the support of the SGs we consider. ($K=5$ for 5-feasible cuts considered by ABC)

G_i be the set of ~~sg~~ SGs at round i . L be the set of simple library gates.

Example: ($K=3$) i.e., we are interested in SGs with at most 3-ips

Let $L = \{AND, OR\}$

$G_0 = \{x_1, x_2, x_3\}$, where x_i are elementary functions.

TT of x_1 is 01010101, x_2 is 00110011, and x_3 is 00001111

G_{i+1} contains G_i and functions of the form $AND(y_1, y_2)$, $OR(y_1, y_2)$ where $y_1, y_2 \in G_i$

Thus, x_1 , $AND(x_1, x_2)$, $AND(x_2, x_1)$, $AND(x_1, x_3)$ e.t.s belongs to G_1

Similarly, $AND(AND(x_1, x_2), x_3)$, $AND(OR(x_2, x_1), AND(x_1, x_3))$ $AND(AND(x_1, x_2), AND(x_1, x_3))$ are some functions in G_2

Problem:- Generating SGs as above generates a large no. of SGs and some of SGs are sub-optimal i.e., they are dominated by other gates. Several pruning techniques are applied.

1) Pruning by dominance :- Whenever a new SG is generated, it is checked against existing SGs that implements the same function (by using a hash table).

Ex:- SG $AND(AND(x_1, x_2), AND(x_1, x_3))$ has worse delay and area than SG $AND(AND(x_1, x_2), x_3)$

2) Pruning by resource limit. Above SG generation suffers from combinatorial explosion. For example, if at one point there are 1000 SGs and a 4-ip NAND is used as a root gate, there would be 1000^4 SGs to consider.

Heuristics to reduce combinatorial explosion

Use only small support gates (≤ 3) as root-gates.

Set area and delay limits on SGs

Idea: Study cut functions actually encountered during mapping and generate good supergates for commonly occurring functions.